

# Full-Stack Personal Portfolio Website

|                     |                                                          |
|---------------------|----------------------------------------------------------|
| Project Assignment: | Personal Portfolio Website Implementation                |
| Architecture:       | MERN Stack (MongoDB, Express.js, React.js, Node.js)      |
| Purpose:            | Internship Project & GitHub Showcase Repository          |
| Documentation Type: | Complete System Architecture & Source Code Specification |

## 1. Project Overview & Key Features

This document provides a comprehensive, deployment-ready implementation blueprint for a production-grade **Full-Stack Personal Portfolio Website**. Built using modern architectures, it showcases practical skills across frontend interface creation, RESTful API design, schema modeling, and robust application hosting.

### Core Key Features Implemented:

- **Frontend:** Highly modular user interface built using React.js, structured to highlight bio, skill sets, dynamic project arrays, and an interactive contact interface.
- **Backend:** RESTful API engine powered by Node.js and Express.js providing robust routing and unified data handshakes.
- **Database:** Scalable relational/non-relational architecture utilizing MongoDB with explicit schema layers definition.
- **Form Processing & Persistence:** Fully dynamic contact system that captures user inquiries, processes them through explicit endpoints, and logs them systematically into the database.

## 2. System Architecture & Database Design

The application is decoupled into a clear client-server architecture model ensuring optimal separation of concerns:



### Database Schema Specifications

Two essential structures are defined to manage data dynamically: **Project** (to drive portfolio elements) and **ContactMessage** (to capture communication leads).

| Schema Collection | Field Name   | Data Type       | Constraints / Description             |
|-------------------|--------------|-----------------|---------------------------------------|
| <b>Projects</b>   | _id          | ObjectId        | Auto-generated Primary Key            |
|                   | title        | String          | Required, Unique, Trimmed             |
|                   | description  | String          | Required                              |
|                   | technologies | Array (Strings) | Required (e.g., ["React", "Node.js"]) |
| <b>Messages</b>   | _id          | ObjectId        | Auto-generated Primary Key            |
|                   | name         | String          | Required, Trimmed                     |
|                   | email        | String          | Required, Validated Format Validation |
|                   | message      | String          | Required                              |
|                   | createdAt    | Date            | Defaults to current timestamp         |

### 3. Backend Source Code (Node.js & Express)

Below is the modularized codebase structure optimized for rapid deployment and absolute reliability.

### 3.1 Server Entrypoint ( `server.js` )

```
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
require('dotenv').config();

const projectRoutes = require('./routes/projectRoutes');
const contactRoutes = require('./routes/contactRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Database Connection
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connection established successfully.'))
  .catch(err => console.error('MongoDB database connection error:', err));

// Routing Bindings
app.use('/api/projects', projectRoutes);
app.use('/api/contact', contactRoutes);

// Root Fallback
app.get('/', (req, res) => {
  res.json({ message: "Portfolio Core REST API Active." });
});

app.listen(PORT, () => {
  console.log(`Server running smoothly on port: ${PORT}`);
});
```

### 3.2 Database Models ( `models/Portfolio.js` )

```
const mongoose = require('mongoose');

// Project Structure Schema
const ProjectSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  technologies: [{ type: String, required: true }],
  liveUrl: { type: String },
  githubUrl: { type: String }
}, { timestamps: true });

// Contact Inquiries Schema
const ContactSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, trim: true, match: [/.\+\@.\+\.\+/, 'Valid email required'] },
  message: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

const Project = mongoose.model('Project', ProjectSchema);
const Contact = mongoose.model('Contact', ContactSchema);

module.exports = { Project, Contact };
```

### 3.3 Core Endpoint Routing & Business Logic

Handling content sourcing and asynchronous message dispatch safely:

```

// routes/projectRoutes.js
const express = require('express');
const router = express.Router();
const { Project } = require('../models/Portfolio');

// Fetch all projects
router.get('/', async (req, res) => {
  try {
    const projects = await Project.find().sort({ createdAt: -1 });
    res.status(200).json(projects);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch portfolio data items.' });
  }
});

module.exports = router;

// routes/contactRoutes.js
const express = require('express');
const router = express.Router();
const { Contact } = require('../models/Portfolio');

// Handle contact form submission
router.post('/', async (req, res) => {
  const { name, email, message } = req.body;
  if (!name || !email || !message) {
    return res.status(400).json({ error: 'Please populate all structural form parameters.' });
  }
  try {
    const structuralInquiry = new Contact({ name, email, message });
    await structuralInquiry.save();
    res.status(201).json({ success: true, message: 'Message logged and persisted successfully!' });
  } catch (err) {
    res.status(500).json({ error: 'Database pipeline capture execution error.' });
  }
});

module.exports = router;

```

## 4. Frontend Interface Layer (React.js)

A declarative React component implementing hooks to bind to the remote API layer cleanly.

## Main Portfolio Wrapper Component ( App.js )

```

import React, { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [projects, setProjects] = useState([]);
  const [formData, setFormData] = useState({ name: '', email: '', message: '' });
  const [statusMessage, setStatusMessage] = useState('');

  // Hydrate data arrays on structural component initialization
  useEffect(() => {
    fetch('https://your-portfolio-backend.herokuapp.com/api/projects')
      .then(res => res.json())
      .then(data => setProjects(data))
      .catch(err => console.error('Error hydrating dynamic projects layout:', err));
  }, []);

  const handleInputChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleFormSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch('https://your-portfolio-backend.herokuapp.com/api/contact', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData)
      });
      const resData = await response.json();
      if (resData.success) {
        setStatusMessage('Thank you! Your inquiry was recorded successfully.');
```

```

        setFormData({ name: '', email: '', message: '' });
      } else {
        setStatusMessage('Error: ' + resData.error);
      }
    } catch (error) {
      setStatusMessage('Network layer processing error. Please try again.');
```

```

    }
  };

  return (
    <div className="portfolio-container">
      <header className="hero-section">
        <h1>Welcome to My Software Engineering Portfolio</h1>
        <p>Full-Stack Engineer specialized in developing scalable distributed web
ecosystems.</p>
      </header>
    </div>
  );
}

```

```

<section className="projects-grid-section">
  <h2>Featured Projects Showcase</h2>
  <div className="projects-wrapper">
    {projects.map((project) => (
      <div key={project._id} className="project-card">
        <h3>{project.title}</h3>
        <p>{project.description}</p>
        <div className="tech-pills">
          {project.technologies.map((tech, idx) => (
            <span key={idx} className="tech-pill">{tech}</span>
          ))}
        </div>
      </div>
    ))}
  </div>
</section>

<section className="contact-form-section">
  <h2>Get In Touch</h2>
  <form onSubmit={handleSubmit} className="contact-form">
    <input type="text" name="name" placeholder="Full Name" value={formData.name}
onChange={handleInputChange} required />
    <input type="email" name="email" placeholder="Email Address"
value={formData.email} onChange={handleInputChange} required />
    <textarea name="message" placeholder="Project Description or Inquiry"
value={formData.message} onChange={handleInputChange} required></textarea>
    <button type="submit">Send Secure Message</button>
  </form>
  {statusMessage && <p className="status-feedback">{statusMessage}</p>}
</section>
</div>
);
}

export default App;

```

## 5. GitHub Repository Blueprint

To make this code GitHub-ready and professional, organize your directory architecture following this explicit layout:



```
my-portfolio-project/
├── backend/
│   ├── config/
│   ├── models/
│   │   └── Portfolio.js
│   ├── routes/
│   │   ├── projectRoutes.js
│   │   └── contactRoutes.js
│   ├── .env
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── App.js
│   │   ├── App.css
│   │   └── index.js
│   └── package.json
└── README.md
```

## 6. Production Deployment Strategy

### 6.1 Backend API Engine Hosting

- **Platforms:** Render, Heroku, or digital instances.
- **Steps:** Connect your GitHub repository to your target platform, link environmental configurations like `MONGO_URI` inside their control dashboard, and define the runtime startup hook command script as `node backend/server.js`.

### 6.2 Frontend Dynamic Application Hosting

- **Platforms:** Vercel, Netlify, or GitHub Pages.
- **Steps:** Run production compilation via `npm run build` locally or configure build workflows using `CI/CD Pipelines` targeting base output directory paths pointing to public assets. Set your environment production endpoints to resolve dynamically to the deployed backend live base URL string format safely.